Light

Programming — Data Structures

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Data types and casting
2. Arrays
3. Records and applying structures
Worked examples
Quick reference
Typical pitfalls

OCR GCSE Computer Science — **Programming: Data Structures (J277/02 Section 2.2)**. This topic covers the fundamental data types (integer, real, Boolean, character, string), how to convert between them (casting), and data structures including one-dimensional arrays, two-dimensional arrays, and records.

Exam questions typically ask you to **identify** appropriate data types, **write pseudocode** using arrays, **trace through** code involving data structures, and **explain** why certain data types or structures are suitable for given scenarios. Questions range from 1-mark identification to 6-mark program writing.

What you must know

This checklist covers the key exam-visible concepts. You should be able to explain each point and demonstrate it with code.

- **Identify and use** the five main data types: integer, real, Boolean, character, and string.
- **Explain** why choosing the correct data type matters for validation and operations.
- **Cast** (convert) between data types using functions like `int()`, `str()`, `real()`.
- **Declare and use** one-dimensional arrays with correct indexing (starting from 0).
- **Declare and use** two-dimensional arrays using row and column indices.
- **Traverse** arrays using loops to access, modify, or search for elements.

- **Explain** what records are and when they are useful for grouping related data.
- **Choose** appropriate data types and structures for given scenarios.

> **Specification note.** OCR J277 Section 2.2 requires understanding of data types (integer, real, Boolean, character, string), casting between types, and data structures (1D arrays, 2D arrays, records). You must be able to write pseudocode using OCR's Exam Reference Language (ERL) and trace through code involving these structures.

Toolkit

Data types — overview

Data type	Description	Examples	Use cases
Integer	Whole number (positive, negative, or zero)	-5, 0, 42, 1000	Counting, ages, quantities, indices
Real	Number with a decimal point (also called float)	3.14, -0.5, 99.99	Prices, measurements, percentages
Boolean	True or false value only	true, false	Flags, conditions, yes/no choices
Character	A single letter, digit, or symbol	'A', '7', '!'	Grades, initials, single symbols
String	A sequence of characters (text)	"Hello", "AB12", ""	Names, addresses, messages

Casting functions (ERL)

Function	Converts to	Example	Result
`int()`	Integer	`int("42")`	42
`int()`	Integer	`int(3.7)`	3 (truncates, doesn't round)
`str()`	String	`str(42)`	"42"
`real()`	Real (float)	`real("3.14")`	3.14
`bool()`	Boolean	`bool(1)`	true

Array syntax (ERL)

1D array declaration:

```
array names[5] // Creates array with indices 0-4
```

Accessing elements:

```
names[0] = "Alice" names[1] = "Bob" print(names[0]) // Outputs: Alice
```

2D array declaration:

```
array grid[3, 4] // 3 rows, 4 columns (indices 0-2, 0-3)
```

Accessing elements:

```
grid[0, 0] = 1 // Row 0, Column 0 grid[2, 3] = 99 // Row 2, Column 3
print(grid[1, 2]) // Row 1, Column 2
```

Exam tip: Array indices start at 0 in OCR pseudocode. An array of size 5 has indices 0, 1, 2, 3, 4 — there is no index 5. This is a very common source of errors.

1. Data types and casting

Every piece of data in a program has a **data type** that determines what values it can hold and what operations can be performed on it. Choosing the correct data type is essential for programs to work correctly and efficiently.

1.1 The five data types

Integer: Whole numbers without decimal points. Integers can be positive, negative, or zero. Use integers for counting, indexing, and any situation where fractional values don't make sense.

```
age = 16 score = 0 temperature = -5 quantity = 100
```

Real (Float): Numbers that can have decimal points. Use reals for measurements, prices, percentages, and calculations that may produce fractional results.

```
price = 19.99 pi = 3.14159 percentage = 75.5 average = 82.3
```

Boolean: Can only be `true` or `false`. Used for conditions, flags, and yes/no situations. Boolean variables are often named as questions (e.g., `isValid`, `hasWon`).

```
gameOver = false loggedIn = true isEven = (number MOD 2 == 0)
```

Character: A single character — one letter, digit, or symbol. Characters are enclosed in single quotes. Note that '7' (character) is different from 7 (integer).

```
grade = 'A' initial = 'J' symbol = '#' digit = '5' // This is a character, not a number
```

String: A sequence of zero or more characters. Strings are enclosed in double quotes. Strings can contain letters, numbers, spaces, and symbols.

```
name = "Alice Smith" postcode = "SW1A 1AA" empty = "" // Empty string (length 0) message = "Hello, World!"
```

1.2 Why data types matter

Using the correct data type is important because:

- **Operations:** Different types support different operations. You can add integers (5 + 3 = 8) but "adding" strings joins them ("5" + "3" = "53").
- **Validation:** The type determines what values are valid. An age cannot be "sixteen" — it must be 16.
- **Memory:** Different types use different amounts of storage.
- **Accuracy:** Using an integer when you need decimals loses precision (e.g., 7 / 2 = 3, not 3.5).

> **Common mistake:** Input from a user is usually a string by default. If you need to do maths with it, you must cast it to an integer or real first.

1.3 Casting (type conversion)

Casting converts a value from one data type to another. This is essential when combining different types or when user input (always a string) needs to be used as a number.

String to Integer:

```
userInput = input("Enter your age: ") // Returns a string age = int(userInput) // Convert to integer nextYear = age + 1 // Now maths works
```

Integer/Real to String:

```
score = 85 message = "Your score is: " + str(score) print(message) // Outputs: Your score is: 85
```

Real to Integer (truncates):

```
price = 19.99 wholePounds = int(price) // Result: 19 (cuts off decimal)
```

> **Exam tip:** Remember that `int()` truncates (cuts off) the decimal — it doesn't round. `int(3.9)` gives 3, not 4.

1.4 Choosing the right data type

Data	Best type	Reason
Number of students	Integer	Always a whole number
Price of an item	Real	Can have pence (decimals)
Whether logged in	Boolean	Yes or no only
A person's name	String	Text with multiple characters
Exam grade (A-F)	Character	Single letter
Phone number	String	May start with 0, has spaces

Worked Example 1 – Identifying data types

Question: State the most appropriate data type for each of the following: (a) A person's height in metres, (b) Whether a light is on or off, (c) A postcode. [3 marks]

Model answer:
(a) Real — height can include decimal values like 1.75 metres **(1)**
(b) Boolean — only two states: on (true) or off (false) **(1)**
(c) String — postcodes contain letters, numbers, and spaces **(1)**

Worked Example 2 – Why casting is needed

Question: A program asks the user to enter two numbers and then adds them together. Explain why casting is necessary. [2 marks]

Model answer:
User input is always returned as a string data type **(1)**. To perform mathematical addition (rather than string concatenation), the strings must be cast to integers or reals using `int()` or `real()` **(1)**.

Worked Example 3 – Writing code with casting

Question: Write pseudocode that asks the user for a price, adds 20% VAT, and outputs the total. [3 marks]

Model answer:

```
priceInput = input("Enter price: ") // (1) Get input price =
real(priceInput) // (1) Cast to real total = price * 1.2 // (1)
```

```
Calculate with VAT print("Total: " + str(total))
```

Worked Example 4 — Explaining a data type choice

Question: A program stores phone numbers. Explain why String is more appropriate than Integer for this data. [2 marks]

Model answer:
Phone numbers can start with a leading zero (e.g., 07123...) which would be lost if stored as an integer **(1)**. Phone numbers may also contain spaces or other characters for formatting **(1)**.

2. Arrays

An **array** is a data structure that stores multiple values of the same type under a single name. Instead of creating separate variables for each item, you can store them all in one array and access them using an index number.

Why use arrays? Imagine storing test scores for 30 students. Without arrays, you would need 30 separate variables: `score1`, `score2`, `score3`... up to `score30`. This is impractical and makes it nearly impossible to loop through all values. With an array, you use a single name `scores` and access each value by its index: `scores[0]`, `scores[1]`, and so on. This allows you to use loops to process all elements efficiently.

2.1 One-dimensional arrays

A **one-dimensional (1D) array** is like a list of items stored in a row. Each item has an index number starting from 0. All elements in an array must be of the same data type — you cannot mix integers and strings in the same array.

Declaring a 1D array:

```
array scores[5] // Array with 5 elements (indices 0-4) array names[10] //
Array with 10 elements (indices 0-9)
```

Assigning values:

```
scores[0] = 85 scores[1] = 92 scores[2] = 78 scores[3] = 95 scores[4] = 88
```

Visual representation:

Index	0	1	2	3	4
Value	85	92	78	95	88

Accessing elements:

```
print(scores[0]) // Outputs: 85 print(scores[3]) // Outputs: 95 x =
scores[1] + scores[2] // x = 92 + 78 = 170
```

2.2 Traversing a 1D array

Traversing means visiting each element in the array, usually with a loop. This is essential for searching, calculating totals, or displaying all values.

Printing all elements:

```
for i = 0 to 4 print(scores[i]) next i
```

Calculating the total:

```
total = 0 for i = 0 to 4 total = total + scores[i] next i print("Total: "
+ str(total))
```

Finding the maximum value:

```
maximum = scores[0] for i = 1 to 4 if scores[i] > maximum then maximum =
scores[i] endif next i print("Maximum: " + str(maximum))
```

> **Exam tip:** When finding maximum, start with the first element as your initial maximum. When finding a total, start with 0.

2.3 Two-dimensional arrays

A **two-dimensional (2D) array** is like a table or grid with rows and columns. You need two indices to access an element: the row index and the column index.

Declaring a 2D array:

```
array grid[3, 4] // 3 rows, 4 columns // Row indices: 0, 1, 2 // Column
indices: 0, 1, 2, 3
```

Assigning values:

```
grid[0, 0] = 1 // Row 0, Column 0 grid[0, 1] = 2 // Row 0, Column 1
grid[1, 2] = 5 // Row 1, Column 2 grid[2, 3] = 9 // Row 2, Column 3
```

Visual representation:

	Col 0	Col 1	Col 2	Col 3
Row 0	1	2	3	4
Row 1	5	6	7	8
Row 2	9	10	11	12

Accessing elements:

```
print(grid[0, 0]) // Outputs: 1 print(grid[1, 2]) // Outputs: 7
print(grid[2, 3]) // Outputs: 12
```

2.4 Traversing a 2D array

To traverse a 2D array, you need nested loops — an outer loop for rows and an inner loop for columns.

Printing all elements:

```
for row = 0 to 2 // 3 rows (0, 1, 2) for col = 0 to 3 // 4 columns (0, 1, 2, 3) print(grid[row, col]) next col next row
```

Calculating total of all elements:

```
total = 0 for row = 0 to 2 for col = 0 to 3 total = total + grid[row, col] next col next row
```

Real-world examples of 2D arrays:

- A seating plan (rows and seats)
- A game board (like noughts and crosses)
- A spreadsheet (rows and columns of data)
- Pixel data in an image (x and y coordinates)

Worked Example 5 — 1D array declaration and access

Question: Write pseudocode to create an array of 4 names, assign values, and print the third name. [3 marks]

Model answer:

```
array names[4] // (1) Declare array names[0] = "Alice" names[1] = "Bob" names[2] = "Charlie" // (1) Assign values names[3] = "Diana" print(names[2]) // (1) Print third name (index 2)
```

Exam tip: The "third" name is at index 2 because indexing starts at 0. First = 0, Second = 1, Third = 2.

Worked Example 6 — Traversing to find a total

Question: An array `prices` contains 6 prices. Write pseudocode to calculate and output the total of all prices. [3 marks]

Model answer:

```
total = 0 // (1) Initialise total for i = 0 to 5 // (1) Loop
through array total = total + prices[i] next i print("Total: " +
str(total)) // (1) Output result
```

Worked Example 7 – 2D array access

Question: A 2D array `seating[5, 8]` represents a theatre with 5 rows and 8 seats per row. Write pseudocode to set seat 3 in row 2 as "booked" and output the value. [2 marks]

Model answer:

```
seating[2, 3] = "booked" // (1) Row 2, Seat 3 print(seating[2,
3]) // (1) Output the value
```

Exam tip: Remember: row comes before column in 2D array notation: `array[row, column]`.

Worked Example 8 – 2D array traversal

Question: Write pseudocode to count how many seats in a 3×4 seating array are marked as "available". [4 marks]

Model answer:

```
count = 0 // (1) Initialise counter for row = 0 to 2 // (1) Loop
through rows for col = 0 to 3 // (1) Loop through columns if
seating[row, col] == "available" then count = count + 1 endif
next col next row print("Available seats: " + str(count)) // (1)
Output
```

3. Records and applying structures

While arrays store multiple values of the same type, sometimes you need to group different types of related data together. **Records** allow you to do this.

3.1 What are records?

A **record** is a data structure that groups together related data items of different types under a single name. Each item in the record is called a **field**. Unlike arrays (which store multiple items of the same type), records can store different data types together.

Why use records? Consider storing information about a student: their name, age, average grade, and enrolment status. You could use four separate variables, but they wouldn't be linked together. You could use parallel arrays (one for names, one for ages, etc.), but this is error-prone and hard to maintain. A record keeps all related data together as a single unit.

For example, a student record might contain:

- **name** (string)
- **age** (integer)
- **averageGrade** (real)
- **isEnrolled** (Boolean)

Defining a record structure:

```
record Student name : String age : Integer averageGrade : Real isEnrolled
: Boolean endrecord
```

Creating and using a record:

```
// Create a new student record student1.name = "Alice Smith" student1.age
= 16 student1.averageGrade = 82.5 student1.isEnrolled = true // Access
record fields print(student1.name) // Outputs: Alice Smith
print(student1.averageGrade) // Outputs: 82.5
```

3.2 Arrays of records

You can combine arrays and records to store multiple records. This is useful for lists of structured data like a class of students or a product inventory.

```
array students[30] // Array of 30 student records students[0].name =
"Alice" students[0].age = 16 students[1].name = "Bob" students[1].age = 15
// Print all student names for i = 0 to 29 print(students[i].name) next i
```

3.3 Choosing the right structure

Different scenarios require different data structures:

Scenario	Best structure	Reason
List of 10 test scores	1D array	Single list of same type values
Seating plan (rows/cols)	2D array	Grid structure with two dimensions
One person's details	Record	Different types of related data
List of products with names and prices	Array of records	Multiple items, each with multiple fields

Scenario	Best structure	Reason
Single value like age	Variable	Just one value — array is overkill

3.4 Common array operations

Here are patterns you should know for working with arrays. These appear frequently in exam questions and form the building blocks for more complex algorithms.

Linear search in an array:

```
found = false position = -1 for i = 0 to 9 if names[i] == searchTerm then
found = true position = i endif next i
```

Counting occurrences:

```
count = 0 for i = 0 to 9 if grades[i] == "A" then count = count + 1 endif
next i
```

Calculating an average:

```
total = 0 for i = 0 to 9 total = total + scores[i] next i average = total
/ 10
```

Finding minimum value:

```
minimum = scores[0] // Start with first element for i = 1 to 9 if
scores[i] < minimum then minimum = scores[i] endif next i
```

Exam tip: In exam questions, pay attention to whether you need a 1D array (simple list), 2D array (grid/table), or record (different data types grouped together).

Worked Example 9 — Record definition and use

Question: A library system needs to store book information including title (text), pages (whole number), and price (decimal). Define an appropriate record structure and show how to create a book. [4 marks]

Model answer:

```
record Book // (1) Define record title : String // (1)
Appropriate types pages : Integer price : Real endrecord
```

```
book1.title = "Python Basics" // (1) Create instance book1.pages
= 250 book1.price = 19.99 // (1) Assign values
```

Worked Example 10 — Searching an array

Question: Write pseudocode to search an array of 20 names for a specific name and output whether it was found. [4 marks]

Model answer:

```
searchName = input("Enter name to search: ") found = false // (1)
Initialise flag for i = 0 to 19 // (1) Loop through array if
names[i] == searchName then // (1) Check each element found =
true endif next i if found == true then // (1) Output result
print("Name found") else print("Name not found") endif
```

Worked Example 11 — Calculating average from array

Question: An array `temperatures[7]` stores daily temperatures for a week. Write pseudocode to calculate and output the average temperature. [3 marks]

Model answer:

```
total = 0 for i = 0 to 6 // (1) Loop through 7 days total = total
+ temperatures[i] // (1) Sum all temperatures next i average =
total / 7 // (1) Calculate average print("Average: " +
str(average))
```

Worked Example 12 — Extended scenario (6-mark style)

Question: A game uses a 3×3 grid for noughts and crosses. Write pseudocode to: (a) declare the 2D array, (b) set all cells to empty ("-"), (c) place an "X" at row 1, column 2. [6 marks]

Model answer:

```
// (a) Declare the 2D array array board[3, 3] // (1) 3 rows, 3
columns // (b) Set all cells to empty for row = 0 to 2 // (1)
Outer loop for rows for col = 0 to 2 // (1) Inner loop for
```

```
columns board[row, col] = "-" // (1) Set to empty next col next
row // (c) Place an X at row 1, column 2 board[1, 2] = "X" // (1)
Correct indices // Output the board (optional, for verification)
for row = 0 to 2 // (1) Display grid print(board[row, 0] + " " +
board[row, 1] + " " + board[row, 2]) next row
```

Worked examples summary

The 12 worked examples above cover the main question types you will encounter on data types and structures. Here is a summary of the key question patterns:

Example	Question type	Key skill
1	Identify types (3 marks)	Match data to appropriate type
2	Explain (2 marks)	Why casting is needed for input
3	Write code (3 marks)	Input, cast, calculate, output
4	Explain choice (2 marks)	String vs Integer for phone numbers
5	Array basics (3 marks)	Declare, assign, access 1D array
6	Traversal (3 marks)	Loop to calculate total
7	2D array access (2 marks)	Row and column indexing
8	2D traversal (4 marks)	Nested loops with condition
9	Record definition (4 marks)	Define structure and create instance
10	Linear search (4 marks)	Search array with flag variable
11	Calculate average (3 marks)	Sum then divide by count
12	Extended scenario (6 marks)	2D array for game board

Quick reference

Data types summary

Type	Description	Example values
Integer	Whole number	-5, 0, 42
Real	Decimal number	3.14, -0.5, 99.99
Boolean	True or false	true, false
Character	Single character	'A', '7', '!'

Type	Description	Example values
String	Text (sequence of characters)	"Hello", "AB12"

Array syntax quick reference

1D Array:

```
array myArray[10] // Declare myArray[0] = value // First element
myArray[9] = value // Last element
```

2D Array:

```
array grid[3, 4] // 3 rows, 4 cols grid[0, 0] = value // Top-left grid[2,
3] = value // Bottom-right
```

Casting functions

Function	Purpose	Example
`int(x)`	Convert to integer	`int("42")` → 42
`str(x)`	Convert to string	`str(42)` → "42"
`real(x)`	Convert to real	`real("3.14")` → 3.14

Common patterns

Sum all elements:

```
total = 0 for i = 0 to n-1 total = total + arr[i] next i
```

Find maximum:

```
max = arr[0] for i = 1 to n-1 if arr[i] > max then max = arr[i] endif next
i
```

Typical pitfalls

Pitfall 1 — Array index out of bounds.
Wrong: `array scores[5]` then accessing `scores[5]`
Right: Array of size 5 has indices **0 to 4**. Index 5 doesn't exist.

Pitfall 2 — Forgetting to cast user input.

Wrong: `age = input("Enter age")` then `age + 1`
Right: Cast first: `age = int(input("Enter age"))`

Pitfall 3 — Confusing string concatenation with addition.
Wrong: `"5" + "3"` = 8
Right: `"5" + "3"` = "53" (strings are joined, not added)

Pitfall 4 — Wrong 2D array indexing order.
Wrong: Thinking `grid[column, row]`
Right: In ERL, it's `grid[row, column]`

Pitfall 5 — Using integer when real is needed.
Wrong: Storing price as Integer (loses pence)
Right: Use Real for any data that can have decimals.

Pitfall 6 — Forgetting int() truncates, not rounds.
Wrong: Thinking `int(3.9)` = 4
Right: `int(3.9)` = 3 (decimal part is cut off)

Pitfall 7 — Not initialising total before summing.
Wrong: Starting loop without `total = 0`
Right: Always initialise accumulator variables before the loop.

Pitfall 8 — Wrong loop bounds for array size.
Wrong: `for i = 0 to 10` for array of size 10
Right: `for i = 0 to 9` (size 10 means indices 0-9)

Pitfall 9 — Using wrong data type for phone numbers.
Wrong: Integer (loses leading zeros)
Right: String (preserves all characters including leading zeros)

Pitfall 10 — Confusing record fields with array indices.
Wrong: `student[name]`
Right: Use dot notation: `student.name`

Assessment Hub | Long-Answer | Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.